

1 Roteiro Prático 04 — Arquitetura Frontend Modular: ES6 Modules & Padrão em Camadas

Objetivos

Objetivos da Atividade (Módulo 2 — ES6 Modules & Padrão em Camadas)

Ao final desta prática orientada em laboratório, o estudante será capaz de:

- Superar a arquitetura monolítica de arquivo único (`app.js`), organizando o frontend em múltiplos módulos especializados utilizando **ES6 Modules** (`import` e `export`);
- Declarar o ponto de entrada modular na página web através da diretiva:
`<script type="module" src="js/main.js"></script>`;
- Compreender e aplicar a separação estrita de responsabilidades no cliente em três camadas:
 - **Camada de Serviços (`services/`):** Regras de negócio, cálculos e persistência pura no `localStorage` sem acoplamento ao DOM;
 - **Camada de Visão (`views/`):** Renderização de cards, tabelas, modais e Toasts via *template literals* do Bootstrap 5;
 - **Camada Utilitária (`utils/`):** Funções puras reutilizáveis de formatação (moeda brasileira, telefone, máscaras);
 - **Orquestrador Central (`main.js`):** Captura de eventos do usuário e coordenação entre camadas.
- Implementar o **CRUD Completo** (*Create, Read, Update, Delete*) com persistência em armazenamento local;
- Implementar **validação visual no cliente** com feedback imediato ao usuário utilizando as classes nativas do Bootstrap 5 (`.is-valid`, `.is-invalid` e `.invalid-feedback`);
- Disparar notificações flutuantes dinâmicas (**Bootstrap Toasts**) para confirmar operações de cadastro, alteração e exclusão;
- Implementar filtragem reativa em tempo real por categoria de serviço na vitrine comunitária.

1.1 Introdução e Contextualização do Projeto

Nos Roteiros 1, 2 e 3, trabalhamos com scripts JavaScript concentrados em um único arquivo (`app.js`). Embora essa abordagem seja excelente para aprender os fundamentos, ela se torna inviável em aplicações reais de médio e grande porte, onde a mistura de seletores do DOM, regras de validação e

persistência gera o chamado “*código espaguete*”.

No **Roteiro Prático 04**, damos o passo definitivo rumo à arquitetura profissional de software no frontend. Desenvolveremos a **Vitrine Comunitária de Empreendedores Locais de Garopaba**, uma aplicação que permite cadastrar, listar, filtrar, editar e gerenciar serviços da comunidade local.

Este projeto serve também como a **base tecnológica inicial para a Ação Curricularizada de Extensão na SNCT (Semana Nacional de Ciência e Tecnologia)**.

1.2 Estrutura Arquitetural de Pastas

O projeto deve ser organizado na pasta `roteiro-4/exercicio-roteiro4/` seguindo rigorosamente a estrutura modular:

Árvore de Diretórios Modular do Projeto:

```
roteiro-4/exercicio-roteiro4/
|-- index.html      <-- Vitrine e forms (BS5)
|-- styles.css      <-- Estilos e transições
`-- js/
    |-- main.js      <-- Orquestrador central
    |-- services/
    |   `-- vitrineService.js <-- CRUD Storage
    |-- views/
    |   `-- vitrineView.js    <-- HTML dinâmico
    `-- utils/
        `-- formatters.js     <-- Formatações
```

1.3 Fundamentação Teórica: Componentes Modernos de Interface (Bootstrap 5)

Antes de iniciarmos a implementação prática guiada, é fundamental compreender a finalidade, o comportamento e a anatomia dos principais componentes de interface gráfica utilizados na aplicação:

1.3.1 1. Janelas Modais (*Modals*): Diálogos Sobrepostos de Alta Atenção

Uma **janela modal** (ou diálogo modal) é um elemento flutuante que surge acima de todo o conteúdo da página, bloqueando temporariamente a interação com a tela de fundo por meio de uma camada escurecida semi-transparente chamada *backdrop*.

- **Finalidade de Uso:** Concentrar a atenção exclusiva do usuário em uma tarefa crítica (como preencher o formulário de cadastro ou atualizar dados de um serviço) sem sair da tela atual nem recarregar o navegador (*Single Page Application*);
- **Anatomia no Bootstrap 5:**
 - `.modal`: Contêiner externo invisível por padrão (`display: none`) com suporte a transição animada suave (`.fade`);

- `.modal-dialog` e `.modal-dialog-centered`: Define a largura, o comportamento responsivo e o alinhamento centralizado vertical na tela;
- `.modal-content`: Invólucro branco do diálogo com bordas arredondadas e sombra (`.shadow-lg`), estruturado em três seções obrigatórias:
 1. `.modal-header`: Cabeçalho com o título da ação (`.modal-title`) e o botão de fechar nativo (`.btn-close` com `data-bs-dismiss="modal"`);
 2. `.modal-body`: Corpo central onde residem os campos do formulário, rótulos e seletores;
 3. `.modal-footer`: Rodapé com botões de ação e submissão (*Cancelar* e *Salvar*).
- **Controle via JavaScript:** O Bootstrap disponibiliza a classe `bootstrap.Modal`, permitindo instanciar e controlar o diálogo programaticamente (`bootstrap.Modal.getInstance(el)`), abrindo com `.show()` e fechando automaticamente após o salvamento com `.hide()`.

1.3.2 2. Notificações Flutuantes (*Toasts*): Feedback Assíncrono Não-Intrusivo

O componente **Toast** é uma notificação compacta e temporária que surge sobreposta em uma posição fixa da tela (canto superior direito) para confirmar o sucesso de operações (ex: *"Serviço cadastrado com sucesso!"*).

- **Vantagem sobre o `alert()`:** O diálogo nativo `window.alert()` é síncrono e bloqueante: ele congela a execução de qualquer outro script JavaScript no navegador e força o usuário a clicar em "OK" para continuar. O **Toast**, por outro lado, é assíncrono, suave e desaparece automaticamente após alguns segundos (*autohide*), proporcionando uma experiência de usuário profissional;
- **Posicionamento com `.toast-container`:** O elemento contêiner utiliza classes utilitárias de posicionamento fixo (`position-fixed top-0 end-0 p-3`) com índice z prioritário (`z-index: 1090`), garantindo visibilidade imediata sem deslocar os demais elementos do layout.

1.3.3 3. Cartões de Conteúdo (*Cards*): Unidades Modulares e Grid Uniforme

Um **Card** é um contêiner retangular flexível (`.card .shadow-sm`) que agrupa de forma coesa todas as informações e ações pertinentes a um único registro:

- `.card-body`: Área principal contendo a *badge* da categoria, o nome do empreendimento em destaque, o preço base formatado, a descrição e o bairro;
- `.card-footer`: Rodapé transparente (`.bg-transparent`) que agrupa as ações disponíveis: link direto para o WhatsApp do empreendedor, botão de edição e botão de exclusão;
- `.h-100`: Classe utilitária fundamental que garante que todos os cards de uma mesma linha tenham exatamente a mesma altura vertical, preservando o alinhamento visual uniforme independentemente do tamanho do texto da descrição.

1.3.4 4. Validação Visual e o Campo Oculto (type="hidden")

- **Atributo novalidate:** Informa ao navegador para suprimir os balões de validação nativos, permitindo que a aplicação assuma o controle visual das mensagens de erro utilizando as classes `.is-valid` (sucesso, verde) e `.is-invalid` (erro, vermelho) combinadas com contêineres contextuais `.invalid-feedback`;
- **Papel Arquitetural do Campo Oculto (#servicoId):** O campo com `type="hidden"` não é renderizado visualmente para o usuário, mas é o elemento-chave que diferencia as operações de **Criação** (*Create*) e **Atualização** (*Update*) no padrão CRUD: se o campo estiver vazio, o clique em "Salvar" cria um novo serviço; se contiver um ID, atualiza o serviço correspondente mantendo o identificador original.

1.4 Fundamentação Teórica: Arquitetura Modular com ES6 Modules

1.4.1 1. Declaração no HTML (type="module")

Ao declarar `<script type="module" src="js/main.js"></script>`, o navegador aplica comportamentos modernos essenciais:

- **Modo Estrito Automático ("use strict"):** Impede o uso de variáveis não declaradas e falhas silenciosas;
- **Escopo Fechado de Módulo:** Variáveis e funções criadas dentro de um arquivo não poluem o escopo global (window);
- **Carregamento Assíncrono com Adiamento (defer):** O script aguarda o parsing completo do DOM antes de executar.

1.4.2 2. Exportação e Importação (export e import)

Para compartilhar recursos entre arquivos, usamos a sintaxe explícita de módulos:

Exemplo de Exportação Nomeada e Importação:

```
// No arquivo js/utils/formatters.js
export function formatarTelefone(telefone) {
  return telefone.replace(
    /^(\d{2})(\d{5})(\d{4})/,
    '($1) $2-$3'
  );
}

// No arquivo js/main.js
import { formatarTelefone }
  from './utils/formatters.js';

// Resultado: (48) 99999-8888
```

```
console.log(formatarTelefone('48999998888'));
```

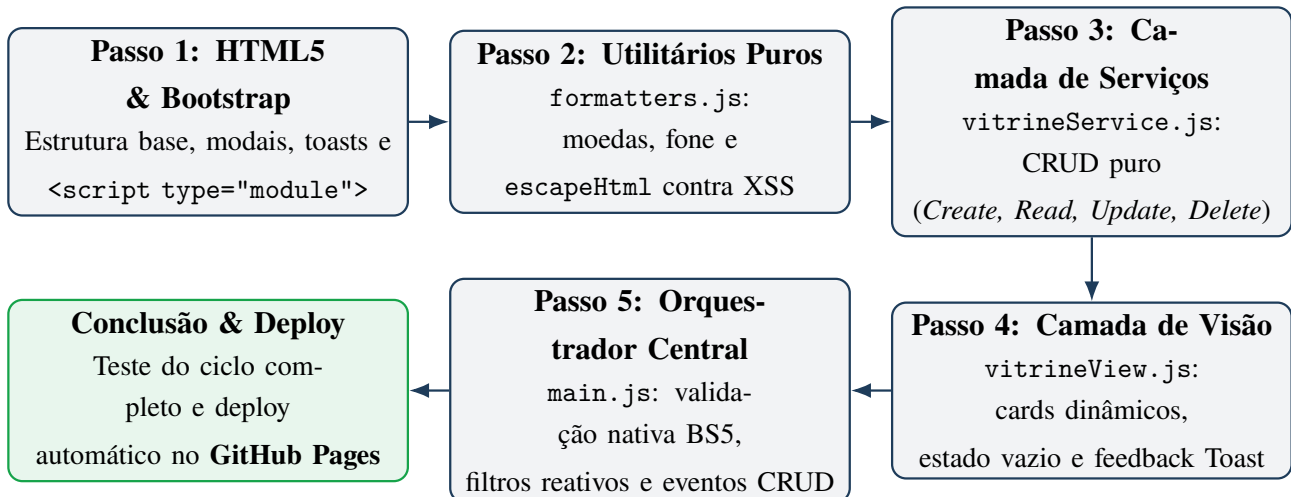


Figura 1: Pipeline arquitetural e sequência cronológica de implementação do Roteiro 04.

1.5 Passo a Passo Guiado do Roteiro

Fase 1.1/5: Estrutura HTML5 Base e Ponto de Entrada (index.html)

Responsabilidade Arquitetural: Estabelecer o esqueleto visual da página, carregar as dependências de estilo (Bootstrap 5 e Icons), fornecer os pontos de ancoragem da interface (`#vitrineContainer`, `#toastNotificacao`, `#totalServicosBadge` e `#filtroCategoria`) e conectar o módulo raiz `js/main.js`.



```
<!-- index.html (Parte 1: Estrutura Base e Scripts) -->
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Vitrine Comunitária de Garopaba - Roteiro 04</title>
  <!-- Bootstrap 5 CSS & Icons CDN -->
  <link rel="stylesheet"
    href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.
      min.css">
  <link rel="stylesheet"
    href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap
      -icons.min.css">
  <link rel="stylesheet" href="styles.css">
</head>
<body class="bg-light">

  <!-- Container de Toasts com corpo #toastMensagem -->
  <div class="toast-container position-fixed top-0 end-0 p-3" style="z-index
    : 1090;">
    <div id="toastNotificacao"
      class="toast align-items-center text-bg-success border-0 shadow"
      role="alert">
      <div class="d-flex">
        <div class="toast-body d-flex align-items-center gap-2">
          <i class="bi bi-check-circle-fill"></i>
        <div>

```

```

        <strong id="toastTitulo" class="d-block">Sucesso!</strong>
        <span id="toastMensagem">Operação realizada com êxito.</span>
    </div>
</div>
<button type="button" class="btn-close btn-close-white me-2 m-auto"
    data-bs-dismiss="toast"></button>
</div>
</div>
</div>

<!-- Header / Barra Superior com Contador e Botão de Ação -->
<header class="bg-success text-white py-3 mb-4 shadow-sm">
    <div class="container d-flex justify-content-between align-items-center
        flex-wrap gap-2">
        <div>
            <h1 class="h4 mb-0 fw-bold">Vitrine de Garopaba</h1>
            <small class="opacity-75">Serviços e comércio locais</small>
        </div>
        <div class="d-flex align-items-center gap-2">
            <span class="badge bg-light text-dark" id="totalServicosBadge">0
                cadastrados</span>
            <button class="btn btn-warning fw-bold" data-bs-toggle="modal"
                data-bs-target="#modalCadastro">
                <i class="bi bi-plus-circle me-1"></i> Divulgar
            </button>
        </div>
    </div>
</header>

<main class="container mb-5">
    <!-- Barra de Filtros com todas as Categorias -->
    <div class="card border-0 shadow-sm p-3 mb-4 bg-white">
        <div class="row align-items-center">
            <div class="col-md-5">
                <select class="form-select" id="filtroCategoria">
                    <option value="todas">Todas as Categorias</option>
                    <option value="Alimentação">Alimentação</option>
                    <option value="Tecnologia">Tecnologia</option>
                    <option value="Artesanato">Artesanato</option>
                    <option value="Serviços Gerais">Serviços Gerais</option>
                    <option value="Turismo">Turismo</option>
                </select>
            </div>
        </div>
    </div>

    <!-- Ponto de Montagem dos Cards Dinâmicos -->
    <div class="row" id="vitrineContainer"></div>
</main>

<!-- Bootstrap Bundle JS + Entrada Modular ES6 -->
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/
    bootstrap.bundle.min.js"></script>
<script type="module" src="js/main.js"></script>
</body>
</html>

```

Checkpoint de Validação 1.1: Como Testar no Navegador

Abra o arquivo `index.html` no navegador usando a extensão **Live Server** do VS Code:

1. A barra verde superior com o título, o badge **"0 cadastrados"** e o botão amarelo **"Divulgar"** devem estar visíveis;
2. O seletor de filtros com as 6 opções disponíveis (Todas + 5 categorias) deve permitir seleção;
3. A área central estará em branco (onde os cards serão injetados pela Camada de Visão).

Nota: Ao clicar em "Divulgar", o modal ainda não abrirá porque será estruturado no Passo 1.2.

Fase 1.2/5: Formulário Modal de Cadastro e Edição (index.html)

Responsabilidade Arquitetural: Fornecer a interface de diálogo acessível para inserção e atualização de serviços, contendo validações visuais nativas do HTML5 (required), seletores padronizados e um campo oculto (#servicoId) que diferencia operações de *Create* (novo) e *Update* (edição).

Onde e como inserir este código no index.html:

- **Atenção:** Não apague nem substitua o código estruturado no Passo 1.1! Todo o documento construído anteriormente permanece intacto.
- **Local exato de inserção:** Insira o bloco completo do modal logo após a tag de fechamento `</main>` e imediatamente antes das tags de script (`<!-- Bootstrap Bundle JS... -->`).

http://127.0.0.1:5500/index.html Navegador Web: index.html (Diálogo Modal Ativo)

Cadastrar Serviço na Vitrine [×]

Nome do Empreendimento: *

Categoria: * **Bairro: ***

Preço Base (R\$): * **WhatsApp / Telefone: ***

Descrição do Serviço: *

Campo Oculto de Controle: `<input type="hidden" id="servicoId">` (não visível, gerencia Create vs Update)

```
<!-- index.html (Inserir logo após </main> e imediatamente antes de <script> ) -->
<!-- =====
-->
<!-- PARTE 2: MODAL DE CADASTRO E EDIÇÃO (Não apaga a Parte 1!)
-->
<!-- =====
-->
<div class="modal fade" id="modalCadastro" tabindex="-1">
  <div class="modal-dialog modal-dialog-centered">
    <div class="modal-content shadow-lg border-0">
      <div class="modal-header">
        <h5 class="modal-title fw-bold" id="modalCadastroLabel">
          Cadastrar Serviço na Vitrine
```

```
</h5>
<button type="button" class="btn-close" data-bs-dismiss="modal"></button>
</div>
<form id="formCadastro" novalidate>
  <div class="modal-body">
    <!-- Campo oculto para ID na operação Update -->
    <input type="hidden" id="servicoId" value="">

    <div class="mb-3">
      <label for="nome" class="form-label small fw-bold">Nome do
        Empreendimento *</label>
      <input type="text" id="nome" class="form-control" required>
    </div>

    <div class="row g-2 mb-3">
      <div class="col-md-6">
        <label for="categoria" class="form-label small fw-bold">
          Categoria *</label>
        <select id="categoria" class="form-select" required>
          <option value="">Selecione...</option>
          <option value="Alimentação">Alimentação</option>
          <option value="Tecnologia">Tecnologia</option>
          <option value="Artesanato">Artesanato</option>
          <option value="Serviços Gerais">Serviços Gerais</option>
          <option value="Turismo">Turismo</option>
        </select>
      </div>
      <div class="col-md-6">
        <label for="bairro" class="form-label small fw-bold">Bairro *<
          /label>
        <input type="text" id="bairro" class="form-control" required>
      </div>
    </div>

    <div class="row g-2 mb-3">
      <div class="col-md-6">
        <label for="precoBase" class="form-label small fw-bold">Preço
          Base (R$) *</label>
        <input type="number" id="precoBase" class="form-control"
          min="0" step="0.5" required>
      </div>
      <div class="col-md-6">
        <label for="telefone" class="form-label small fw-bold">
          WhatsApp / Telefone *</label>
        <input type="tel" id="telefone" class="form-control"
          placeholder="48999998888" required>
      </div>
    </div>

    <div class="mb-3">
      <label for="descricao" class="form-label small fw-bold">Descriçã
        o do Serviço *</label>
      <textarea id="descricao" class="form-control" rows="3" required>
      </textarea>
    </div>
  </div>
</form>
```

```
<div class="modal-footer">
  <button type="button" class="btn btn-light" data-bs-dismiss="modal
    ">Cancelar</button>
  <button type="submit" id="btnSalvarModal" class="btn btn-success
    fw-bold">
    <span id="btnSalvarTexto">Salvar na Vitrine</span>
  </button>
</div>
</form>
</div>
</div>
</div>
```

Checkpoint de Validação 1.2: Como Testar a Abertura do Modal

Atualize o navegador e clique no botão amarelo **”Divulgar”**:

1. O modal deve abrir de forma suave e centralizada na tela;
2. Verifique se os 6 campos possuem seus rótulos e se o select de categoria exibe todas as opções;
3. Clique em **”Cancelar”** ou no botão **”X”** para confirmar que o fechamento nativo do Bootstrap funciona.

Fase 2/5: Utilitários Puros (js/utils/formatters.js)

Responsabilidade Arquitetural: Funções puras e reutilizáveis de transformação de dados. Recebem entradas brutas e retornam formatos amigáveis para o usuário brasileiro (moeda e telefone), além de realizar a sanitização de strings contra ataques XSS (*Cross-Site Scripting*). Não tocam no DOM nem no armazenamento.

Croqui Visual 2: Fluxo Puro de Entrada e Saida

```
[ "48999998888" ] -----> [ formatarTelefone() ] -----> "(48)
99999-8888"
[ 35.5 ] -----> [ formatarMoeda() ] -----> "R$ 35,50"
[ "<script>..." ] -----> [ escapeHtml() ] -----> "&lt;script&gt;
;..."
```

```
// js/utils/formatters.js
export function formatarMoeda(valor) {
  const num = parseFloat(valor) || 0;
  return num.toLocaleString('pt-BR', {
    style: 'currency',
    currency: 'BRL'
  });
}

export function formatarTelefone(fone) {
  const digits = (fone || '').replace(/\D/g, '');
  if (digits.length === 11) {
    return digits.replace(
      /^(\d{2})(\d{5})(\d{4})/,
      '($1) $2-$3'
    );
  }
  return fone;
}

export function escapeHtml(texto) {
  const div = document.createElement('div');
  div.textContent = texto;
  return div.innerHTML;
}
```

Checkpoint de Validação 2: Teste via Console do Navegador (F12)

Abra o navegador, pressione **F12** (ou botão direito → Inspecionar) e selecione a aba **Console**. Execute:

```
import('./js/utils/formatters.js').then(m =>
  console.log(m.formatarMoeda(149.9)));
```

Resultado esperado no Console: "R\$ 149,90". Isso comprova que a exportação nomeada e o cálculo nativo estão 100% operacionais!

Fase 3/5: Camada de Serviços e Dados (js/services/vitrineService.js)

Responsabilidade Arquitetural: Isolar completamente a regra de persistência e ciclo de vida do CRUD (*Create, Read, Update, Delete*). Opera estritamente sobre a chave 'garopaba_vitrine_servicos' do localStorage do navegador, sem gerar nenhuma tag HTML nem manipular o DOM.

Croqui Visual 3: Arquitetura da Camada de Serviços

```
[ Interface / main.js ] <--(obter, salvar, atualizar, remover)--> [ vitrineService ]
```

|
v
[LocalStorage (Browser)]

```
// js/services/vitrineService.js
const STORAGE_KEY = 'garopaba_vitrine_servicos';

export function obterServicos() {
  const dados = localStorage.getItem(STORAGE_KEY);
  return dados ? JSON.parse(dados) : [];
}

export function salvarServico(novo) {
  const servicos = obterServicos();
  const completo = {
    id: Date.now().toString(),
    dataCadastro: new Date().toISOString(),
    ...novo
  };
  servicos.unshift(completo);
  localStorage.setItem(STORAGE_KEY, JSON.stringify(servicos));
  return completo;
}

export function atualizarServico(id, dados) {
  const servicos = obterServicos();
  const idx = servicos.findIndex(s => s.id === id);
  if (idx !== -1) {
    servicos[idx] = { ...servicos[idx], ...dados,
      dataEdicao: new Date().toISOString() };
    localStorage.setItem(STORAGE_KEY, JSON.stringify(servicos));
    return servicos[idx];
  }
  return null;
}

export function removerServico(id) {
  const servicos = obterServicos().filter(s => s.id !== id);
  localStorage.setItem(STORAGE_KEY, JSON.stringify(servicos));
}
```

```
    return servicos;  
}
```

Checkpoint de Validação 3: Teste de Persistência no Console (F12)

No Console do navegador (**F12**), teste a persistência de um serviço:

```
import('./js/services/vitrineService.js').then(s => {  
  s.salvarServico({ nome: 'Oficina Garopaba', categoria: 'Serviços  
    Gerais', precoBase: 50 });  
  console.log('Serviços no LocalStorage:', s.obterServicos());  
});
```

Resultado esperado: O console imprimirá um array com o serviço recém-criado, contendo um identificador único id gerado automaticamente. Verifique também na aba **Application** → **Local Storage**.

Fase 4/5: Camada de Visão (js/views/vitrineView.js)

Responsabilidade Arquitetural: Construir e manipular os elementos visuais na interface. Transforma objetos de dados em cards HTML (criarCardHtml), renderiza coleções no container com tratamento de estado vazio (renderizarCards) e dispara mensagens de notificação flutuantes (exibirToast).

Alimentação

R\$ 25,00

Padaria Central da Ferrugem

Bairro: Ferrugem | ID: 1729482103

Pães de fermentação natural, café orgânico especial e tortas artesanais com entrega local.

WhatsApp: (48) 99999-8888

Editar

Excluir

```
// js/views/vitrineView.js (Parte 1: Template do Card)
import { formatarMoeda, formatarTelefone, escapeHtml }
  from '../utils/formatters.js';

export function criarCardHtml(s) {
  const tel = (s.telefone || '').replace(/\\D/g, '');
  return `
    <div class="col-md-6 col-lg-4 mb-4">
      <div class="card h-100 shadow-sm border-0 vitrine-card">
        <div class="card-body">
          <div class="d-flex justify-content-between mb-2">
            <span class="badge bg-success">
              ${escapeHtml(s.categoria)}
            </span>
            <span class="fw-bold text-primary">
              ${formatarMoeda(s.precoBase)}
            </span>
          </div>
          <h5 class="card-title fw-bold mb-1">${escapeHtml(s.nome)}</h5>
          <h6 class="text-muted small mb-3">
            <i class="bi bi-geo-alt-fill text-danger me-1"></i>
            ${escapeHtml(s.bairro)}
          </h6>
          <p class="card-text text-secondary small">
            ${escapeHtml(s.descricao)}
          </p>
        </div>
        <div class="card-footer bg-transparent border-0 pt-0 pb-3
          d-flex justify-content-between align-items-center">
          <a href="https://wa.me/55${tel}" target="_blank"
            class="btn btn-sm btn-outline-success">
            <i class="bi bi-whatsapp me-1"></i>
            ${formatarTelefone(s.telefone)}
          </a>
        </div>
      </div>
    </div>`
}
```

```
        <div class="d-flex gap-1">
          <button class="btn btn-sm btn-outline-primary btn-editar"
            data-id="${s.id}" title="Editar">
            <i class="bi bi-pencil-square"></i>
          </button>
          <button class="btn btn-sm btn-outline-danger btn-excluir"
            data-id="${s.id}" title="Excluir">
            <i class="bi bi-trash"></i>
          </button>
        </div>
      </div>
    </div>
  </div>`;
}

// js/views/vitrineView.js (Parte 2: Renderização e Estado Vazio)
export function renderizarCards(servicos, container) {
  if (!servicos || servicos.length === 0) {
    container.innerHTML = `
      <div class="col-12 text-center py-5">
        <i class="bi bi-inbox fs-1 text-muted"></i>
        <h5 class="text-muted mt-2">
          Nenhum serviço cadastrado nesta categoria.
        </h5>
      </div>`;
    return;
  }
  container.innerHTML = servicos.map(criarCardHtml).join('');
}

// js/views/vitrineView.js (Parte 3: Notificações Flutuantes Toasts)
export function exibirToast(mensagem, tipo = 'success') {
  const toastEl = document.getElementById('toastNotificacao');
  const toastBody = document.getElementById('toastMensagem');
  if (toastEl && toastBody) {
    toastBody.innerText = mensagem;
    toastEl.className = `toast align-items-center text-bg-${tipo} border-0`;
    const toast = bootstrap.Toast.getOrCreateInstance(toastEl);
    toast.show();
  }
}
```

Checkpoint de Validação 4: Teste de Renderização e Toast no Console

No Console do navegador (**F12**), teste o disparo da notificação:

```
import('./js/views/vitrineView.js').then(v => v.exibirToast('Notificação funcionando!'));
```

Resultado esperado: O Toast flutuante verde surgirá no canto superior direito da tela!

Fase 5/5: Orquestrador Central (js/main.js)

Responsabilidade Arquitetural: O controlador principal da aplicação. Importa os serviços e as visões, escuta o carregamento do DOM, conecta os eventos de submissão do formulário (*Create & Update*), a delegação de cliques nos cards (*Edit & Delete*) e a reatividade instantânea dos filtros.

Croqui Visual 5: Ciclo de Vida da Orquestracao

Usuario clica/submete ---> main.js valida ---> vitrineService (localStorage)
|
Interface renderiza cards <--- vitrineView <-----+ (atualizarInterface)

```
// js/main.js
import {
  obterServicos, salvarServico,
  atualizarServico, removerServico
} from './services/vitrineService.js';
import {
  renderizarCards, exibirToast
} from './views/vitrineView.js';

document.addEventListener('DOMContentLoaded', () => {
  const form = document.getElementById('formCadastro');
  const container = document.getElementById('vitrineContainer');
  const filtroCategoria = document.getElementById('filtroCategoria');
  const contadorEl = document.getElementById('totalServicosBadge');
  const modalEl = document.getElementById('modalCadastro');
  const servicoIdInput = document.getElementById('servicoId');

  function atualizarInterface() {
    const todos = obterServicos();
    const filtro = filtroCategoria.value;
    const filtrados = filtro === 'todas'
      ? todos
      : todos.filter(s => s.categoria === filtro);
    renderizarCards(filtrados, container);
    if (contadorEl) {
      contadorEl.innerHTML = `${todos.length} cadastrados`;
    }
  }

  // Filtros Reativos
  filtroCategoria.addEventListener('change', atualizarInterface);
  atualizarInterface();

  // Submissão do Formulário (Create & Update)
  form.addEventListener('submit', (e) => {
    e.preventDefault();
    if (!form.checkValidity()) {
      e.stopPropagation();
    }
  });
});
```

```
    form.classList.add('was-validated');
    return;
}

const idAtual = servicoIdInput ? servicoIdInput.value : '';
const getVal = (id) => document.getElementById(id).value.trim();

const dados = {
  nome: getVal('nome'),
  categoria: document.getElementById('categoria').value,
  bairro: getVal('bairro'),
  precoBase: parseFloat(getVal('precoBase')) || 0,
  telefone: getVal('telefone'),
  descricao: getVal('descricao')
};

if (idAtual) {
  atualizarServico(idAtual, dados);
  exibirToast('Serviço atualizado com sucesso!');
} else {
  salvarServico(dados);
  exibirToast('Empreendimento cadastrado com sucesso!');
}

form.reset();
if (servicoIdInput) servicoIdInput.value = '';
form.classList.remove('was-validated');

if (modalEl) {
  const modal = bootstrap.Modal.getInstance(modalEl);
  if (modal) modal.hide();
}
atualizarInterface();
});

// Delegação de Cliques nos Cards (Edit & Delete)
container.addEventListener('click', (e) => {
  const btnEditar = e.target.closest('.btn-editar');
  if (btnEditar) {
    const id = btnEditar.getAttribute('data-id');
    const item = obterServicos().find(s => s.id === id);
    if (item) {
      if (servicoIdInput) servicoIdInput.value = item.id;
      document.getElementById('nome').value = item.nome;
      document.getElementById('categoria').value = item.categoria;
      document.getElementById('bairro').value = item.bairro;
      document.getElementById('precoBase').value = item.precoBase;
      document.getElementById('telefone').value = item.telefone;
      document.getElementById('descricao').value = item.descricao;

      if (modalEl) {
        const modal = bootstrap.Modal.getOrCreateInstance(modalEl);
        modal.show();
      }
    }
  }
  return;
})
```

```
const btnExcluir = e.target.closest('.btn-excluir');
if (btnExcluir) {
  const id = btnExcluir.getAttribute('data-id');
  if (confirm('Deseja realmente remover este serviço da vitrine?')) {
    removerServico(id);
    atualizarInterface();
    exibirToast('Serviço removido da vitrine.', 'warning');
  }
}
});
});
```

Checkpoint de Validação 5: Teste Completo da Aplicação Ponta a Ponta

Parabéns! Sua aplicação está 100% integrada. Execute o ciclo completo no navegador:

1. **Create:** Clique em "Divulgar", preencha o formulário e salve. O card deve surgir imediatamente na vitrine e o contador deve somar +1;
2. **Filter:** Mude o filtro de categoria no select. Apenas serviços da categoria escolhida devem permanecer na tela;
3. **Update:** Clique no ícone de lápis em um card. O modal abrirá com os dados carregados. Altere o valor e salve;
4. **Delete:** Clique no ícone de lixeira, confirme o aviso nativo e veja o item desaparecer em tempo real!

1.6 Checklist de Conclusão e Critérios Avaliativos

Antes de realizar o commit final no repositório, certifique-se de validar todos os itens:

- ☐ O projeto utiliza estritamente o padrão ES6 Modules com a tag:
`<script type="module" src="js/main.js"></script>`;
- ☐ A estrutura em pastas (js/services/, js/views/, js/utils/) foi rigorosamente respeitada;
- ☐ O arquivo vitrineService.js opera puramente sobre o localStorage sem manipular elementos do DOM;
- ☐ As quatro operações do CRUD (**Create, Read, Update e Delete**) foram implementadas e testadas com persistência no navegador;
- ☐ O formulário exibe validações visuais nativas do Bootstrap 5 (.is-invalid) quando campos obrigatórios não são preenchidos;
- ☐ A vitrine reage instantaneamente aos filtros de categoria sem recarregar a página;
- ☐ O código está devidamente publicado e em pleno funcionamento no **GitHub Pages**.